

Table of Content

Introduction:	1
Dataset Description:	2
MapReduce Analysis:	3
Pig Analysis:	13
Conclusion:	22

Introduction:

The project focuses on analyzing a large-scale ride-sharing dataset using Apache Pig and Hadoop MapReduce on top of the Hadoop ecosystem. Since real-world datasets are often difficult to access or may lack controlled structure, a synthetic dataset containing 1 million records was generated to simulate realistic ride-sharing operations across multiple cities.

At the core of this system is Hadoop, which provides a distributed framework for storing and processing big data. Hadoop mainly consists of two key components used in this project:

- **HDFS (Hadoop Distributed File System)**

HDFS is responsible for storing the dataset across the system. Instead of keeping data in a single machine, it distributes the file into blocks and stores them across nodes. This ensures scalability and fault tolerance. The dataset (rides_big.csv) is first placed into HDFS so that it can be accessed efficiently by both Pig and MapReduce jobs.

- **MapReduce Framework**

MapReduce is the processing layer of Hadoop. It divides large data into smaller chunks, processes them in parallel using mappers, and then aggregates results using reducers. This allows efficient handling of millions of records.

On top of Hadoop, **Apache Pig** is used as a higher-level abstraction. Pig scripts are automatically converted into MapReduce jobs internally, which means both approaches ultimately run on the same distributed infrastructure (HDFS + MapReduce engine).

The project performs multiple analytical operations on the dataset to extract meaningful insights related to ride-sharing behavior. The analysis includes identifying demand patterns, revenue distribution, payment trends, diversity of ride types, and top-performing drivers.

Peak demand is analyzed by identifying the most active hours for rides in each city. Revenue-related insights are generated by aggregating fare values to understand both total and average earnings. Payment behavior is examined by grouping transactions based on city and payment method, revealing usage patterns across regions. Ride-type diversity is measured using statistical distribution (HHI index) to understand how balanced or concentrated ride options are within each city. Additionally, driver performance is evaluated by calculating total revenue per driver and identifying the top contributors.

Dataset Description:

The dataset used in this project is a **synthetic ride-sharing dataset** generated using a Python script. The purpose of generating synthetic data was to simulate a real-world large-scale ride-sharing system while maintaining control over data distribution and avoiding dependency on external datasets.

The dataset contains **1,000,000 records**, representing individual ride transactions across multiple cities in Bangladesh.

Data Generation Approach

- Data was generated using Python's built-in csv and random libraries.
- Cities were assigned using weighted probability, with Dhaka having the highest frequency to reflect realistic urban demand.
- Ride types were randomly selected, but distance ranges were adjusted based on ride type.
- Fare was calculated using a base rate per ride type $\times$ distance + small random variation.
- Additional attributes such as payment method, hour, and day were randomly distributed.

Dataset Schema

Field Name	Data Type	Description
ride_id	Integer	Unique identifier for each ride
driver_id	Integer	ID of the driver (1–50,000)
user_id	Integer	ID of the user (1–200,000)
city	String	City where the ride occurred (Dhaka, Chittagong, Sylhet, Khulna, Rajshahi)
ride_type	String	Type of ride (Bike, Car, CNG, Premium, Shared)
distance	Float	Distance of the trip in kilometers
fare	Float	Total fare calculated based on distance and ride type
payment_method	String	Payment type (Cash, Card, Bkash, Nagad)
hour	Integer	Hour of the ride (0–23)

day	String	Day of the week (Mon–Sun)
year	Integer	Year of ride (2022 or 2023)

Key Characteristics of the Dataset

- Large-scale: 1 million rows suitable for big data processing (Pig & MapReduce)
- Skewed distribution: Higher concentration of rides in Dhaka
- Realistic fare modeling:
 - Bike → low cost
 - Premium → high cost
- Temporal coverage:
 - Includes both hourly and weekly patterns
- Diverse attributes:
 - Supports multiple types of analysis (revenue, demand, anomalies, etc.)

MapReduce Analysis:

1. Average Fare by Ride Type

Objective:

Returns the average fare for each ride type.

This analysis comprises of :

1. **Driver Class** : Sets up job configuration, mapper and reducer classes, and defines output types
2. **Mapper Class** : Emits ride_type (Text) as key and fare (DoubleWritable) as value
3. **Reducer Class** : Calculates total fare and count, then outputs average fare per ride type

Code Snippet:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```

public class AvgFareByRideType {

    public static class Map extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {

            String[] f = value.toString().split(",");

            if (f.length < 11) return;

            outKey.set(f[4]);

            context.write(outKey, new DoubleWritable(Double.parseDouble(f[6])));

        }

    }

    public static class Reduce extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

        public void reduce(Text key, Iterable<DoubleWritable> values, Context context)

            throws IOException, InterruptedException {

                double sum = 0;

                int count = 0;

                for (DoubleWritable v : values) {

                    sum += v.get();

                    count++;

                }

                context.write(key, new DoubleWritable(sum / count));

            }

    }

    public static void main(String[] args) throws Exception {

        Job job = Job.getInstance(new Configuration(), "Avg Fare");

```

```

job.setJarByClass(AvgFareByRideType.class);

job.setMapperClass(Map.class);

job.setReducerClass(Reduce.class);

job.setOutputKeyClass(Text.class);

job.setOutputValueClass(DoubleWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);

}

}

```

Output Screenshot:

The screenshot displays a web interface for viewing HDFS data. The browser address bar shows `localhost:9870/explorer.html#/AvgFareByRideTypeOutput`. The interface includes a sidebar with navigation options like 'Datanodes', 'Directory', and 'File Output'. The main content area has tabs for 'Download', 'Head the file (first 32K)', and 'Tail the file (last 32K)'. The 'Block information' section for 'Block 0' shows the following details:

- Block ID: 1073741946
- Block Pool ID: BP-586615610-192.168.10.213-1784996577453
- Generation Stamp: 1122
- Size: 119
- Availability:
 - Hameem.lan

The 'File contents' section displays the following text:

```

Bike 51.04210815806616
CNG 111.06704197012138
Car 293.8249275832971
Premium 606.4704501079185
Shared 49.96964697694966

```

2. Payment Distribution by City

Objective:

Counts how many times each payment method is used in each city.

This analysis comprises of :

1. **Driver Class** : Sets up mapper and reducer with (Text, IntWritable) output
2. **Mapper Class** : Emits city_paymentMethod as key and 1 as value
3. **Reducer Class** : Aggregates counts for each key

Code Snippet:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;


public class PaymentByCity {

    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {

            String[] f = value.toString().split(",");

            if (f.length < 11) return;

            outKey.set(f[3] + "_" + f[7]); // city_payment
```

```

        context.write(outKey, one);
    }
}

public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)

        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable v : values) sum += v.get();

        context.write(key, new IntWritable(sum));

    }

}

public static void main(String[] args) throws Exception {

    Job job = Job.getInstance(new Configuration(), "Payment Distribution");

    job.setJarByClass(PaymentByCity.class);

    job.setMapperClass(Map.class);

    job.setReducerClass(Reduce.class);

    job.setOutputKeyClass(Text.class);

    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));

    FileOutputFormat.setOutputPath(job, new Path(args[1]));

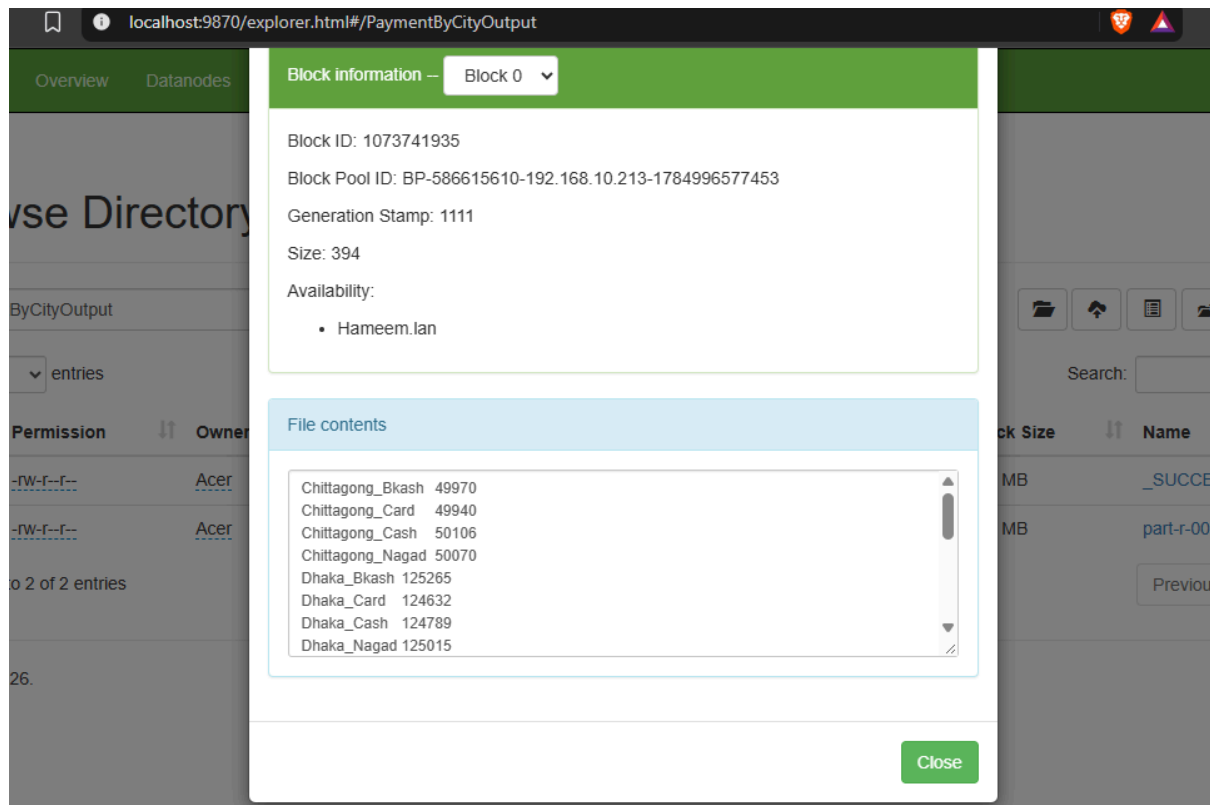
    System.exit(job.waitForCompletion(true) ? 0 : 1);

}

}

```

Output Screenshot:



3. Peak Hour Demand per City

Objective:

Finds ride demand distribution across hours for each city.

This analysis comprises of :

1. **Driver Class :** Configures job execution
2. **Mapper Class :** Emits city_hour as key and 1 as value
3. **Reducer Class :** Sums rides per (city, hour)

Code Snippet:

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```

public class PeakHourPerCity {

    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {

            String[] f = value.toString().split(",");

            if (f.length < 11) return;

            String city = f[3];

            String hour = f[8];

            outKey.set(city + "_" + hour);

            context.write(outKey, one);} }

    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)

            throws IOException, InterruptedException {

            int sum = 0;

            for (IntWritable v : values) sum += v.get();

            context.write(key, new IntWritable(sum));} }

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Peak Hour");

        job.setJarByClass(PeakHourPerCity.class);

        job.setMapperClass(Map.class);

        job.setReducerClass(Reduce.class);

        job.setOutputKeyClass(Text.class);

```

```

job.setOutputValueClass(IntWritable.class);

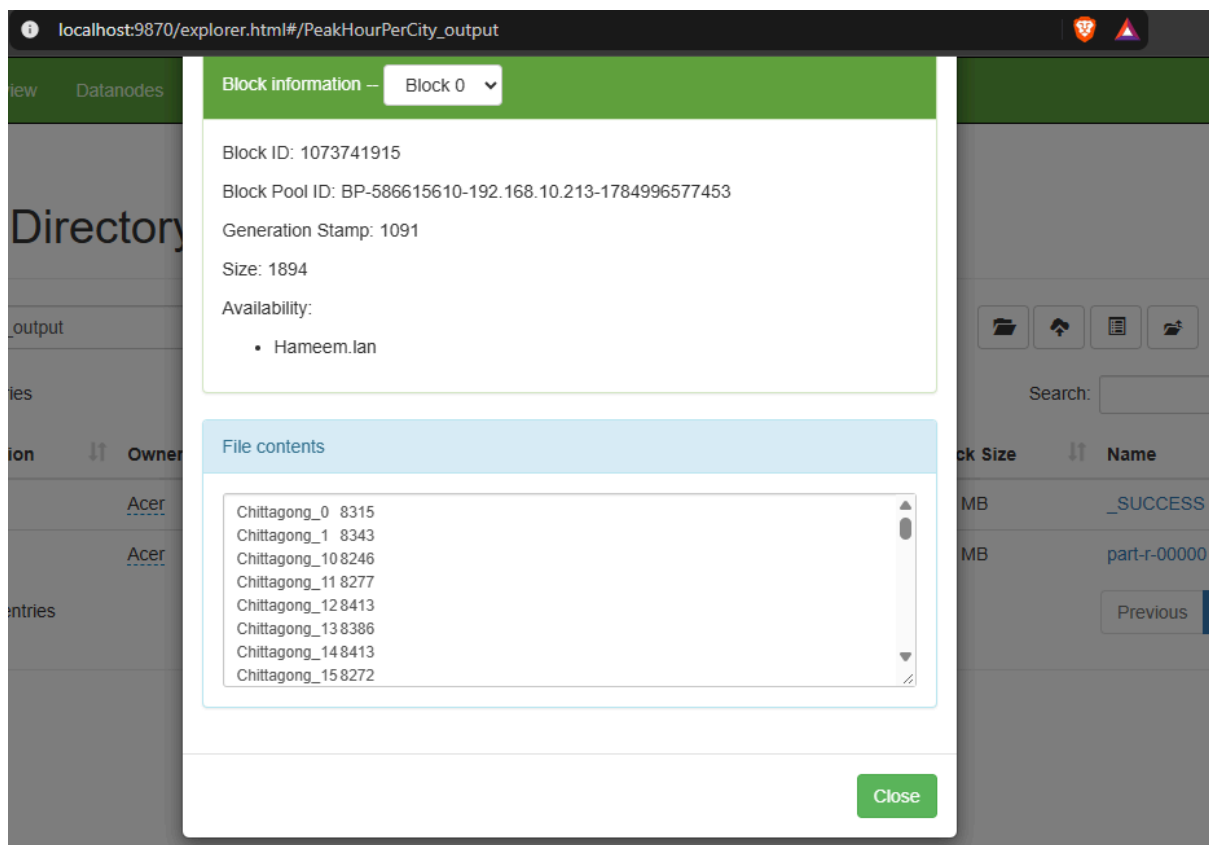
FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);}}

```

Output Screenshot:



4. Top Drivers by Revenue

Objective:

Calculates total revenue generated by each driver.

This analysis comprises of :

1. **Driver Class** : Sets job configuration
2. **Mapper Class** : Emits driver_id as key and fare as value
3. **Reducer Class** : Sums total earnings per driver

Code Snippet:

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TopDriversRevenue {

    public static class Map extends Mapper<LongWritable, Text, Text, DoubleWritable> {

        private Text outKey = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {

            String[] f = value.toString().split(",");

            if (f.length < 11) return;

            outKey.set(f[1]); // driver_id

            double fare = Double.parseDouble(f[6]);

            context.write(outKey, new DoubleWritable(fare)); } }

    public static class Reduce extends Reducer<Text, DoubleWritable, Text, DoubleWritable>
    { public void reduce(Text key, Iterable<DoubleWritable> values, Context context)

        throws IOException, InterruptedException {

            double sum = 0;

            for (DoubleWritable v : values) sum += v.get();

            context.write(key, new DoubleWritable(sum)); } }

    public static void main(String[] args) throws Exception {

        Job job = Job.getInstance(new Configuration(), "Driver Revenue");

        job.setJarByClass(TopDriversRevenue.class);

```

```
job.setMapperClass(Map.class);

job.setReducerClass(Reduce.class);

job.setOutputKeyClass(Text.class);

job.setOutputValueClass(DoubleWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1); }}
```

Output Screenshot:

The screenshot shows a web browser window at `localhost:9870/explorer.html#/TopDriversRevenueOutput`. A modal window titled "Block information" is open, displaying details for "Block 0". The block information includes:

- Block ID: 1073741925
- Block Pool ID: BP-586615610-192.168.10.213-1784996577453
- Generation Stamp: 1101
- Size: 967097
- Availability: Hameem.lan

Below the block information, there is a "File contents" section showing a list of data entries:

10007	4275.96
10008	7494.91
10009	6852.950000000001
1001	4878.6
10010	4551.91
10011	6145.62
10012	5518.71
10013	3297.16

A "Close" button is located at the bottom right of the modal window.

Pig Analysis:

1. Fare per KM Anomaly Detection

- This script calculates fare per kilometer for each ride and then computes the overall average.
- It identifies anomalies by filtering rides where fare per km is more than twice the average, highlighting unusually expensive trips.

Code Snippet:

```
data = LOAD 'rides_big.csv' USING PigStorage(',')
```

```
AS (
```

```
    ride_id:long,
```

```
    driver_id:int,
```

```
    user_id:int,
```

```
    city:chararray,
```

```
    ride_type:chararray,
```

```
    distance:double,
```

```
    fare:double,
```

```
    payment:chararray,
```

```
    hour:int,  day:chararray,  year:int
```

```
);
```

```
calc = FOREACH data GENERATE
```

```
    ride_id,
```

```
    city,
```

```
    ride_type,
```

```
    fare,
```

```
    distance,
```

```
    (fare / distance) AS fare_per_km;
```


2. Payment Method Distribution by City

- Data is grouped by city and payment method to count how often each payment type is used.
- The results are sorted to clearly show dominant payment methods within each city.

Code Snippet:

```
data = LOAD 'rides_big.csv' USING PigStorage(',')
```

```
AS (
```

```
    ride_id:long,
```

```
    driver_id:int,
```

```
    user_id:int,
```

```
    city:chararray,
```

```
    ride_type:chararray,
```

```
    distance:double,
```

```
    fare:double,
```

```
    payment:chararray,
```

```
    hour:int,
```

```
    day:chararray,
```

```
    year:int
```

```
);
```

```
grp = GROUP data BY (city, payment);
```

```
counts = FOREACH grp GENERATE
```

```
    group.city AS city,
```

```
    group.payment AS payment,
```

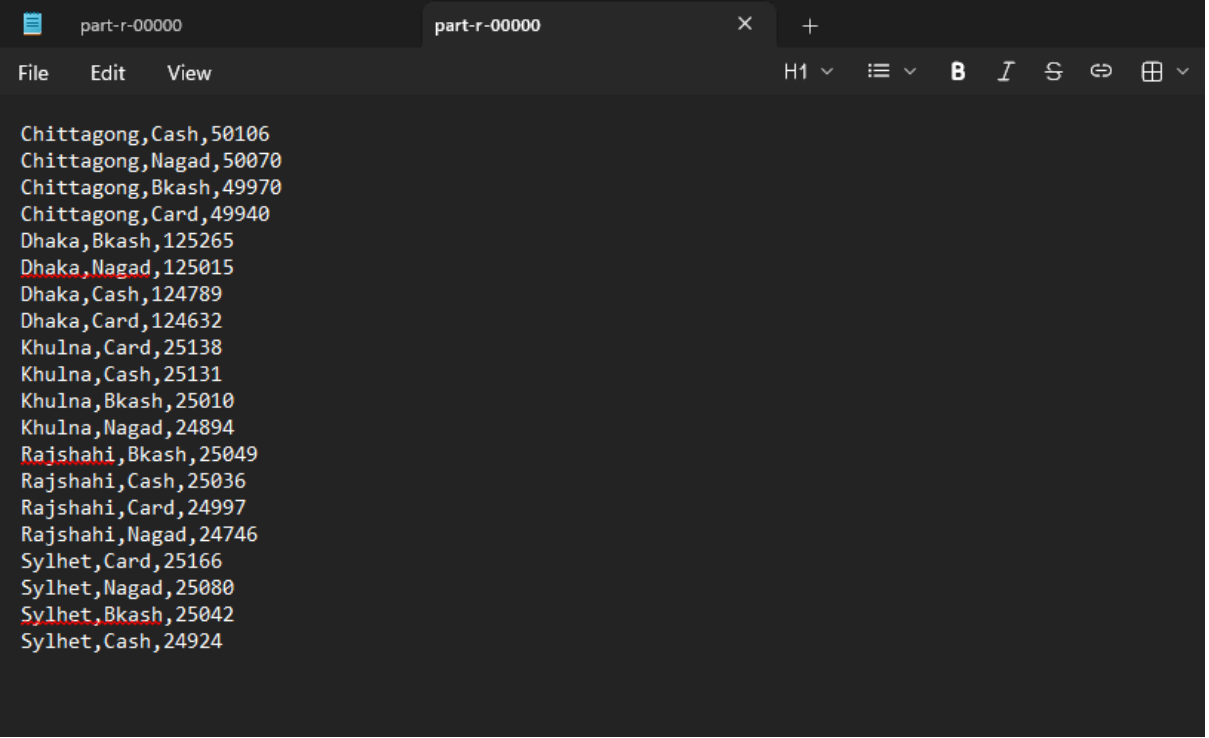
```
    COUNT(data) AS total;
```

```
sorted = ORDER counts BY city, total DESC;
```

```
DUMP sorted;
```

STORE sorted INTO 'output_folder/payment_method_distribution' USING PigStorage(',');

Output Screenshot:



The screenshot shows a text editor window with a dark theme. The title bar indicates the file is 'part-r-00000'. The editor contains a list of ride records, each consisting of a city name, a payment method, and a count, separated by commas. The records are as follows:

City	Payment Method	Count
Chittagong	Cash	50106
Chittagong	Nagad	50070
Chittagong	Bkash	49970
Chittagong	Card	49940
Dhaka	Bkash	125265
Dhaka	Nagad	125015
Dhaka	Cash	124789
Dhaka	Card	124632
Khulna	Card	25138
Khulna	Cash	25131
Khulna	Bkash	25010
Khulna	Nagad	24894
Rajshahi	Bkash	25049
Rajshahi	Cash	25036
Rajshahi	Card	24997
Rajshahi	Nagad	24746
Sylhet	Card	25166
Sylhet	Nagad	25080
Sylhet	Bkash	25042
Sylhet	Cash	24924

3. Peak Hour per City

- Rides are grouped by city and hour to count total trips for each time slot.
- For each city, the hour with the highest ride count is selected as the peak demand period.

Code Snippet:

```
data = LOAD 'rides_big.csv' USING PigStorage(',')
```

```
AS (
```

```
    ride_id:long,
```

```
    driver_id:int,
```

```
    user_id:int,
```

```
    city:chararray,
```

```
    ride_type:chararray,
```

```
    distance:double,
```

```

    fare:double,

    payment:chararray,

    hour:int,

    day:chararray,

    year:int

);

grp = GROUP data BY (city, hour);

counts = FOREACH grp GENERATE

    group.city AS city,

    group.hour AS hour,

    COUNT(data) AS total;

by_city = GROUP counts BY city;

peak = FOREACH by_city {

    sorted = ORDER counts BY total DESC;

    top = LIMIT sorted 1;

    GENERATE FLATTEN(top);

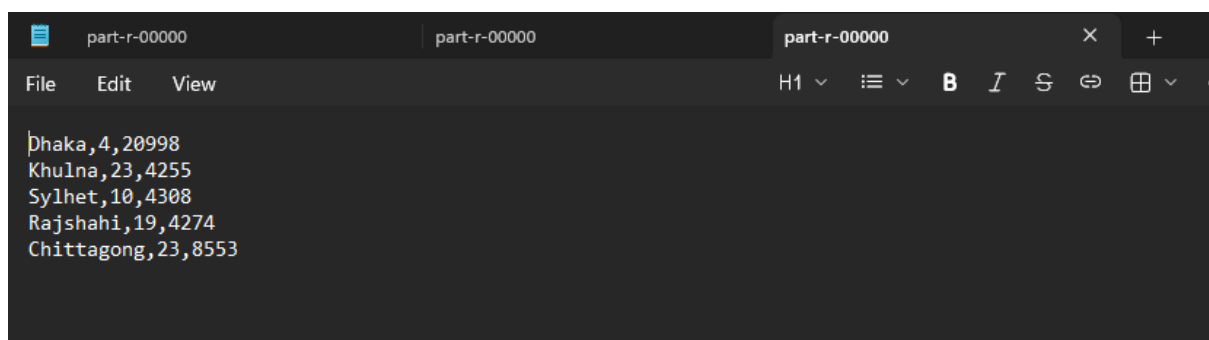
};

DUMP peak;

STORE peak INTO 'output_folder/peak_hour' USING PigStorage(',');

```

Output Screenshot:



```

part-r-00000  part-r-00000  part-r-00000  X  +
File Edit View H1  H1  B I S G  4
Dhaka,4,20998
Khulna,23,4255
Sylhet,10,4308
Rajshahi,19,4274
Chittagong,23,8553

```

4. Ride Type Diversity (HHI Index)

- This analysis measures how diverse ride types are within each city using a concentration metric (HHI).
- A higher value indicates dominance of fewer ride types, while a lower value shows more balanced usage.

Code Snippet:

```
data = LOAD 'rides_big.csv' USING PigStorage(',')
```

```
AS (
```

```
    ride_id:long, driver_id:int,
```

```
    user_id:int,
```

```
    city:chararray,
```

```
    ride_type:chararray,
```

```
    distance:double,
```

```
    fare:double,
```

```
    payment:chararray,
```

```
    hour:int,
```

```
    day:chararray,
```

```
    year:int
```

```
);
```

```
grp = GROUP data BY (city, ride_type);
```

```
counts = FOREACH grp GENERATE
```

```
    group.city AS city,
```

```
    group.ride_type AS ride_type,
```

```
    COUNT(data) AS cnt;
```

```
city_total = GROUP counts BY city;
```

```
totals = FOREACH city_total GENERATE
```

```

    group AS city,

    SUM(counts.cnt) AS total;

joined = JOIN counts BY city, totals BY city;

share = FOREACH joined GENERATE

    counts::city AS city,

    counts::ride_type AS ride_type,

    (double)counts::cnt / totals::total AS p;

sq = FOREACH share GENERATE

    city,

    p * p AS sq_val;

grp2 = GROUP sq BY city;

hhi = FOREACH grp2 GENERATE

    group AS city,

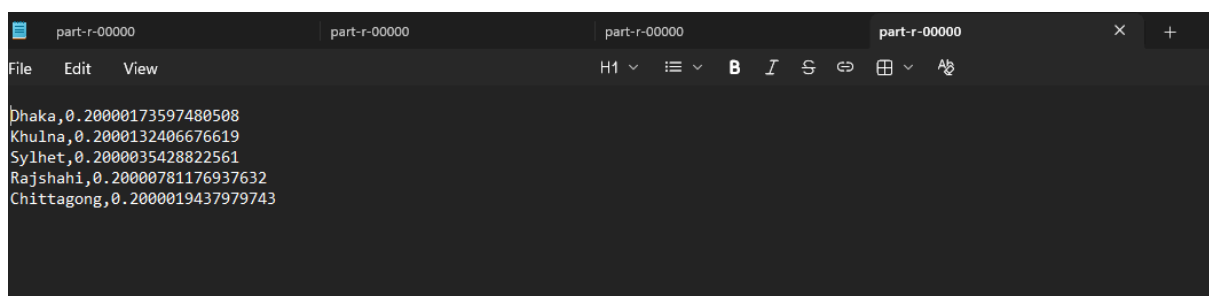
    SUM(sq.sq_val) AS hhi_index;

DUMP hhi;

STORE hhi INTO 'output_folder/ride_type_diversity' USING PigStorage(',');

```

Output Screenshot:



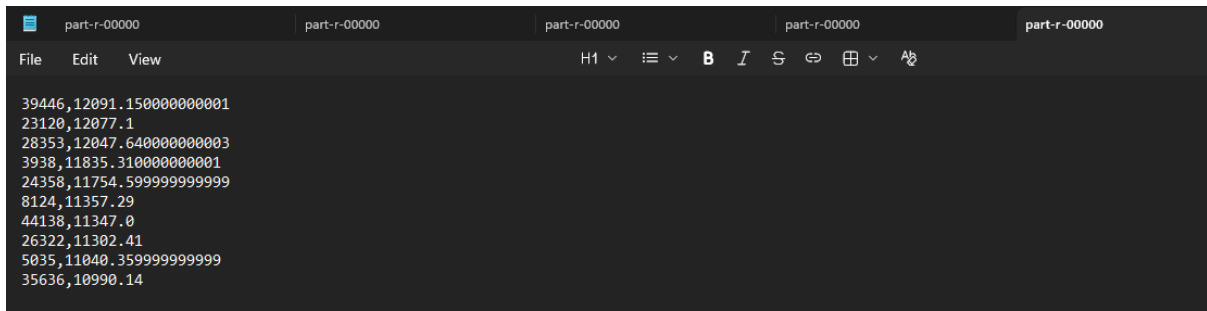
5. Top 10 Drivers by Revenue

- Total revenue is calculated per driver by summing all fares.
- Drivers are sorted by revenue, and the top 10 highest earners are selected.

Code Snippet:

```
data = LOAD 'rides_big.csv' USING PigStorage(',')
AS (
    ride_id:long,
    driver_id:int,
    user_id:int,
    city:chararray,
    ride_type:chararray,
    distance:double, fare:double,
    payment:chararray,
    hour:int, day:chararray, year:int);
grp = GROUP data BY driver_id;
driver_rev = FOREACH grp GENERATE
    group AS driver_id,
    SUM(data.fare) AS total_revenue;
sorted = ORDER driver_rev BY total_revenue DESC;
top10 = LIMIT sorted 10;
DUMP top10;
STORE top10 INTO 'output_folder/top_10_drivers' USING PigStorage(',');
```

Output Screenshot:



```
39446,12091.150000000001
23120,12077.1
28353,12047.640000000003
3938,11835.310000000001
24358,11754.599999999999
8124,11357.29
44138,11347.0
26322,11302.41
5035,11040.359999999999
35636,10990.14
```

Conclusion:

The project successfully demonstrates how large-scale data can be processed and analyzed using both Pig and MapReduce. Each analysis task, such as peak hour detection, revenue calculation, payment distribution, and average fare computation, was implemented using both approaches to understand their differences.

Pig proved to be more concise and easier to write due to its declarative nature, while MapReduce required more detailed implementation but provided deeper insight into how data flows through the system. Despite the difference in complexity, both approaches produced consistent analytical results.

Overall, the work highlights the importance of choosing the right tool based on the use case. Pig is suitable for faster development and simpler workflows, whereas MapReduce is more flexible for customized and performance-critical tasks.